

Java Threading Evolution Tutorial

From Basic Threads → Multithreading → Async Programming

Introduction

This tutorial covers the evolution of concurrent programming in Java:

1. **Basic Threads** (Java 1.0) - Foundation
2. **Runnable Interface** (Java 1.0) - Better design
3. **ExecutorService** (Java 5) - Thread pool management
4. **Future & Callable** (Java 5) - Return values
5. **CompletableFuture** (Java 8) - Modern async
6. **Virtual Threads** (Java 21) - Massive scalability
7. **Spring Boot @Async** - Framework approach

1 Basic Thread (Old Way - Don't Use in Production)

Concept

Extend Thread class and override run() method.

Code Example

```
public class BasicThreadExample {  
    public static void main(String[] args) {  
        // Creating threads by extending Thread class  
        Thread thread1 = new MyThread("Thread-1");  
    }  
}
```

```
Thread thread2 = new MyThread("Thread-2");

thread1.start(); // Starts new thread
thread2.start();

System.out.println("Main thread continues...");
}
}

class MyThread extends Thread {
    private String name;

    public MyThread(String name) {
        this.name = name;
    }

    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(name + ": Count " + i);
            try {
                Thread.sleep(1000); // Pause 1 second
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Output

Main thread continues...

Thread-1: Count 1

Thread-2: Count 1
Thread-1: Count 2
Thread-2: Count 2
...

✓ Pros

- Simple to understand
- Direct control

✗ Cons

- Can't extend other classes (Java single inheritance)
- No thread reuse (creates new thread each time)
- Hard to manage many threads
- No return value from thread

2 Runnable Interface (Better Approach)

Concept

Implement Runnable interface - separates task from thread management.

Code Example

```
public class RunnableExample {  
    public static void main(String[] args) {  
        // Create tasks  
        Runnable task1 = new DatabaseTask("DB-Query-1");  
        Runnable task2 = new DatabaseTask("DB-Query-2");  
  
        // Create threads with tasks  
        Thread thread1 = new Thread(task1);
```

```

Thread thread2 = new Thread(task2);

thread1.start();
thread2.start();

// Lambda syntax (Java 8+)
Thread thread3 = new Thread(() -> {
    System.out.println("Lambda task running");
});
thread3.start();
}
}

class DatabaseTask implements Runnable {
    private String queryName;

    public DatabaseTask(String queryName) {
        this.queryName = queryName;
    }

    @Override
    public void run() {
        System.out.println(queryName + " started by " +
            Thread.currentThread().getName());

        // Simulate database query
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println(queryName + " completed");
    }
}

```

}

✓ Pros

- Can extend other classes
- Task is separate from execution
- Can use lambdas

✗ Cons

- Still manually creating threads
- No thread reuse
- No return value
- Hard to manage lifecycle

3 ExecutorService (Thread Pool Management)

Concept

Reusable thread pool - submit tasks, threads are managed automatically.

Code Example

```
import java.util.concurrent.*;

public class ExecutorServiceExample {
    public static void main(String[] args) {
        // Create thread pool with 3 threads
        ExecutorService executor = Executors.newFixedThreadPool(3);

        // Submit 10 tasks - only 3 run at a time
        for (int i = 1; i <= 10; i++) {
            int taskId = i;
```

```

executor.submit(() -> {
    System.out.println("Task " + taskId +
        " running on " +
        Thread.currentThread().getName());
    try {
        Thread.sleep(2000); // Simulate work
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    System.out.println("Task " + taskId + " completed");
});
}

// Shutdown executor
executor.shutdown();

try {
    // Wait for all tasks to complete
    if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
        executor.shutdownNow();
    }
} catch (InterruptedException e) {
    executor.shutdownNow();
}
}
}

```

Different Thread Pool Types

// Fixed size pool

```
ExecutorService fixed = Executors.newFixedThreadPool(5);
```

// Cached pool - creates threads as needed

```
ExecutorService cached = Executors.newCachedThreadPool();

// Single thread
ExecutorService single = Executors.newSingleThreadExecutor();

// Scheduled tasks
ScheduledExecutorService scheduled =
    Executors.newScheduledThreadPool(2);
scheduled.scheduleAtFixedRate(() -> {
    System.out.println("Runs every 5 seconds");
}, 0, 5, TimeUnit.SECONDS);
```

✓ Pros

- Thread reuse (efficient)
- Automatic thread management
- Queue management
- Better resource control

✗ Cons

- Still no easy way to get return values
- Complex error handling
- Callback hell for chaining tasks

4 Future & Callable (Getting Return Values)

Concept

Callable returns a value, Future holds the result.

Code Example

```
import java.util.concurrent.*;
import java.util.*;

public class FutureExample {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newFixedThreadPool(3);

        // Callable returns a value
        Callable<Integer> task1 = () -> {
            Thread.sleep(2000);
            return 100;
        };

        Callable<String> task2 = () -> {
            Thread.sleep(1000);
            return "Database Result";
        };

        // Submit and get Future objects
        Future<Integer> future1 = executor.submit(task1);
        Future<String> future2 = executor.submit(task2);

        // Do other work while tasks execute
        System.out.println("Tasks submitted, doing other work...");

        // Get results (blocks until ready)
        Integer result1 = future1.get(); // Waits up to 2 seconds
        String result2 = future2.get();

        System.out.println("Result 1: " + result1);
        System.out.println("Result 2: " + result2);
    }
}
```



```
        executor.shutdown();
    }
}
```

Real-World Example: Dashboard API (Like Your Code)

```
public class DashboardService {
    private ExecutorService executor = Executors.newFixedThreadPool(10);

    public DashboardResponse getDashboard() throws Exception {
        // Submit all tasks
        Future<List<Site>> sites =
            executor.submit(() -> fetchSites());

        Future<Integer> contractCount =
            executor.submit(() -> fetchContractCount());

        Future<List<Waste>> wasteData =
            executor.submit(() -> fetchWasteData());

        // Wait for all results
        DashboardResponse response = new DashboardResponse();
        response.setSites(sites.get(5, TimeUnit.SECONDS)); // Timeout
        response.setContractCount(contractCount.get(5, TimeUnit.SECONDS));
        response.setWasteData(wasteData.get(5, TimeUnit.SECONDS));

        return response;
    }

    private List<Site> fetchSites() {
        // Database call
        return siteRepository.findAll();
    }
}
```

```
private Integer fetchContractCount() {  
    return contractRepository.count();  
}  
  
private List<Waste> fetchWasteData() {  
    return wasteRepository.findAll();  
}  
}
```

✓ Pros

- Can return values
- Timeout support
- Exception handling

✗ Cons

- .get() blocks the thread
- No composition (hard to chain)
- Awkward error handling
- Callback hell for complex flows

5 CompletableFuture (Modern Async - Java 8+)

Concept

Non-blocking async programming with composition and chaining.

Basic Example

```
import java.util.concurrent.CompletableFuture;
```

```

public class CompletableFutureExample {
    public static void main(String[] args) {
        // Simple async task
        CompletableFuture<String> future =
            CompletableFuture.supplyAsync(() -> {
                sleep(2000);
                return "Hello";
            });

        // Chain operations
        future
            .thenApply(result -> result + " World") // Transform
            .thenApply(String::toUpperCase)         // Transform again
            .thenAccept(System.out::println)         // Consume
            .exceptionally(ex -> {                   // Handle errors
                System.err.println("Error: " + ex.getMessage());
                return null;
            });

        // Main thread continues
        System.out.println("Non-blocking!");

        // Wait for completion
        future.join();
    }

    static void sleep(long ms) {
        try { Thread.sleep(ms); }
        catch (InterruptedException e) { }
    }
}

```

Combining Multiple Futures

```
public class CombiningFutures {
    public static void main(String[] args) {
        CompletableFuture<Integer> future1 =
            CompletableFuture.supplyAsync(() -> {
                sleep(2000);
                return 100;
            });

        CompletableFuture<Integer> future2 =
            CompletableFuture.supplyAsync(() -> {
                sleep(1000);
                return 200;
            });

        // Combine results
        CompletableFuture<Integer> combined =
            future1.thenCombine(future2, (a, b) -> a + b);

        System.out.println("Sum: " + combined.join()); // 300

        // Wait for all
        CompletableFuture.allOf(future1, future2).join();

        // Wait for any
        CompletableFuture.anyOf(future1, future2).join();
    }

    static void sleep(long ms) {
        try { Thread.sleep(ms); }
        catch (InterruptedException e) { }
    }
}
```

```
}
```

Your Dashboard with CompletableFuture (IMPROVED)

```
@Service
```

```
public class DashboardServiceImproved {
```

```
    // Custom thread pool (IMPORTANT!)
```

```
    private final ExecutorService executor =
```

```
        Executors.newFixedThreadPool(16);
```

```
    public DashboardResponse getDashboard(Long startDate, Long endDate) {
```

```
        // Start all tasks asynchronously
```

```
        CompletableFuture<List<Site>> sitesFuture =
```

```
            CompletableFuture.supplyAsync(
```

```
                () -> fetchSites(),
```

```
                executor
```

```
            ).orTimeout(5, TimeUnit.SECONDS);
```

```
        CompletableFuture<Integer> contractsFuture =
```

```
            CompletableFuture.supplyAsync(
```

```
                () -> fetchContracts(),
```

```
                executor
```

```
            ).orTimeout(5, TimeUnit.SECONDS);
```

```
        CompletableFuture<List<Waste>> wasteFuture =
```

```
            CompletableFuture.supplyAsync(
```

```
                () -> fetchWaste(),
```

```
                executor
```

```
            ).orTimeout(5, TimeUnit.SECONDS);
```

```
        // Wait for all to complete
```

```

CompletableFuture.allOf(
    sitesFuture,
    contractsFuture,
    wasteFuture
).join();

// Build response with error handling
DashboardResponse response = new DashboardResponse();

try {
    response.setSites(sitesFuture.get());
} catch (Exception e) {
    log.error("Failed to fetch sites", e);
    response.setSites(Collections.emptyList());
}

try {
    response.setContractCount(contractsFuture.get());
} catch (Exception e) {
    log.error("Failed to fetch contracts", e);
    response.setContractCount(0);
}

try {
    response.setWasteData(wasteFuture.get());
} catch (Exception e) {
    log.error("Failed to fetch waste", e);
    response.setWasteData(Collections.emptyList());
}

return response;
}

private List<Site> fetchSites() {

```

```
        return siteRepository.findAll();
    }

    private Integer fetchContracts() {
        return contractRepository.count();
    }

    private List<Waste> fetchWaste() {
        return wasteRepository.findAll();
    }

    @PreDestroy
    public void cleanup() {
        executor.shutdown();
    }
}
```

Pros

- Non-blocking
- Easy composition/chaining
- Better error handling
- Timeout support
- Partial failure handling

Cons

- More complex than simple threads
- Need to manage executor
- Learning curve

6 Spring Boot @Async (Framework Way)

Configuration

@Configuration

@EnableAsync

```
public class AsyncConfig {

    @Bean(name = "taskExecutor")
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(10);
        executor.setMaxPoolSize(20);
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("async-");
        executor.initialize();
        return executor;
    }
}
```

Using @Async

@Service

```
public class DashboardServiceAsync {

    @Async("taskExecutor")
    public CompletableFuture<List<Site>> fetchSitesAsync() {
        List<Site> sites = siteRepository.findAll();
        return CompletableFuture.completedFuture(sites);
    }

    @Async("taskExecutor")
    public CompletableFuture<Integer> fetchContractsAsync() {
```



```

Integer count = contractRepository.count();
return CompletableFuture.completedFuture(count);
}

@Async("taskExecutor")
public CompletableFuture<List<Waste>> fetchWasteAsync() {
    List<Waste> waste = wasteRepository.findAll();
    return CompletableFuture.completedFuture(waste);
}

// Controller or Service using these
public DashboardResponse getDashboard() {
    CompletableFuture<List<Site>> sites = fetchSitesAsync();
    CompletableFuture<Integer> contracts = fetchContractsAsync();
    CompletableFuture<List<Waste>> waste = fetchWasteAsync();

    // Wait for all
    CompletableFuture.allOf(sites, contracts, waste).join();

    DashboardResponse response = new DashboardResponse();
    response.setSites(sites.join());
    response.setContractCount(contracts.join());
    response.setWasteData(waste.join());

    return response;
}
}

```

Pros

- Spring manages thread pool
- Clean annotation-based
- Easy to configure

- Integrated with Spring context

✗ Cons

- Must call from different bean (proxy issue)
- Less control than manual approach

7 Virtual Threads (Java 21+ - Future)

Concept

Lightweight threads managed by JVM - can create millions!

Code Example

```
public class VirtualThreadExample {  
    public static void main(String[] args) throws Exception {  
        // Create virtual thread  
        Thread vThread = Thread.ofVirtual().start(() -> {  
            System.out.println("Running on virtual thread");  
        });  
        vThread.join();  
  
        // Execute many tasks  
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {  
            // Can handle millions of these!  
            for (int i = 0; i < 10_000; i++) {  
                int taskId = i;  
                executor.submit(() -> {  
                    System.out.println("Task " + taskId);  
                    Thread.sleep(Duration.ofSeconds(1));  
                    return taskId;  
                });  
            }  
        }  
    }  
}
```

```
    } // Auto-shutdown  
  }  
}
```

Spring Boot with Virtual Threads

@Configuration

```
public class VirtualThreadConfig {
```

```
    @Bean
```

```
    public TomcatProtocolHandlerCustomizer<?>
```

```
protocolHandlerVirtualThreadExecutorCustomizer() {
```

```
    return protocolHandler -> {
```

```
        protocolHandler.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
```

```
    };
```

```
}
```

```
}
```

Pros

- Millions of threads possible
- No thread pool needed
- Perfect for I/O-bound tasks
- Simple mental model

Cons

- Requires Java 21+
- Not for CPU-bound tasks
- Still maturing

Best Practices Summary

1. Choose Right Tool

Simple task → Runnable + ExecutorService

Need result → CompletableFuture

Spring app → @Async

Java 21+ → Virtual Threads

High scale → Reactive (WebFlux)

2. Always Use Thread Pools

//  DON'T

```
CompletableFuture.supplyAsync(() -> task()); // Uses ForkJoinPool
```

//  DO

```
ExecutorService executor = Executors.newFixedThreadPool(10);
```

```
CompletableFuture.supplyAsync(() -> task(), executor);
```

3. Always Set Timeouts

```
future.orTimeout(5, TimeUnit.SECONDS)
    .exceptionally(ex -> handleTimeout(ex));
```

4. Handle Errors Per Future

```
CompletableFuture<Data> future = fetchData()
    .exceptionally(ex -> {
        log.error("Failed", ex);
        return defaultValue();
    });
```

5. Shutdown Executors

```
@PreDestroy
public void cleanup() {
    executor.shutdown();
    try {
        if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
            executor.shutdownNow();
        }
    } catch (InterruptedException e) {
        executor.shutdownNow();
    }
}
```

6. Monitor Thread Pools

```
ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
executor.setThreadNamePrefix("dashboard-");
executor.setRejectedExecutionHandler(new
ThreadPoolExecutor.CallerRunsPolicy());
executor.initialize();
```

Quick Decision Tree

Do you need return value?

- ├ NO → Runnable + ExecutorService
- └ YES → Do you need composition/chaining?
 - ├ NO → Callable + Future
 - └ YES → Do you use Spring?
 - ├ YES → @Async
 - └ NO → CompletableFuture

For 10,000+ concurrent tasks?

└ Use Virtual Threads (Java 21+)

Performance Comparison

Scenario: 1000 database calls

Basic Threads: 1000 threads created → 🚀 OutOfMemoryError

ExecutorService: 10 threads reused → ✅ 100s

CompletableFuture: 10 threads + non-blocking → ✅ 100s

Virtual Threads: 1000 virtual threads → ✅ 10s

Need help implementing any of these in your specific dashboard code? Let me know!